Reviewer Pack — Cauchy Mean Width of Minterm Hull Lower-Bounds Monotone k-CL…

Entry 65d1ffdf3cc6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 01:28:31 UTC

Cauchy Mean Width of Minterm Hull Lower-Bounds Monotone k-CLIQUE

Entry ID: `65d1ffdf3cc6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-17 01:28:31 UTC

1. Conjecture

Field A (mathematical branch): Convex geometry (Cauchy mean-width / intrinsic 1-volume of polytopes in R^N, in the spirit of Schneider's Brunn–Minkowski theory, rarely connected to circuit complexity) **Field B** (complexity object): Karchmer–Wigderson / monotone DNF size and formula complexity for the k-CLIQUE indicator

Statement:

> For a monotone Boolean function f on N variables, let M(f) ⊂ {0,1}^N be its set of minterms (minimal satisfying assignments viewed as 0/1 vectors) and let K(f) := conv(M(f) ∪ {0}) ⊆ [0,1]^N. Define μ(f) := MW(K(f))^2, where MW(K) = 2·E_{u~Unif(S^{N−1})}[max_{x∈K}⟨u,x⟩] is the Cauchy mean width. We conjecture: (i) for every monotone DNF representation of f with s terms, μ(f) ≤ C_1·log(s+1)·(log N+1); (ii) for the k-CLIQUE indicator on K_v with k=⌈$\log_2$ v⌉ (so N = v(v−1)/2), μ(f) ≥ C_2·v, for absolute constants C_1,C_2>0. A single instance with μ(f) > C_1·log(s+1)(log N+1), or a k-CLIQUE indicator with μ < C_2·v at v ≤ 9, falsifies the conjecture.

Rationale (proposer's reasoning):

> Monotone DNFs are unions of axis-aligned subcubes, so conv(M(f)) is the convex hull of at most s lattice vectors of bounded coordinate sum, and its mean width should grow only logarithmically with the number of generators (a Sudakov/Talagrand-style upper bound for sparse 0/1 hulls). The k-CLIQUE minterm cloud, in contrast, lives on the (k choose 2)-th slice and is rotationally rich because v-vertex automorphisms act transitively on edges, forcing the supporting hyperplanes to spread evenly over many directions and inflating mean width linearly in v. If both bounds hold, any monotone DNF for k-CLIQUE has s ≥ 2^{Ω(v/log v)}, a Razborov-flavor separation via a purely metric-geometric witness that bypasses approximator combinatorics.

Taxonomy category: `KARCHMER_WIGDERSON` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1f6d4a0de5280cbf`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across 30 seeds for each (s,N) with s∈{5,10,20,40}, N∈{12,18,24,28}, every DNF must satisfy μ(f) ≤ 2·[3·log(s+1)(log N+1)]. For v∈{4..9}, k=⌈log₂ v⌉, the OLS regression of μ vs v must have slope ≥ 0.5 and R^2 ≥ 0.9, with μ(f) ≥ 0.5·v/2 per v.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): `- mean width polytope monotone circuit complexity lower bound - Cauchy mean width minterms DNF size clique - intrinsic volume Boolean function complexity k-clique monotone`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=33.9s

5.1 Generated Python source

```python
import sys
import random
import math
import json
from itertools import combinations
from collections import defaultdict

def generate_minterms(dnf, N):
    minterms = []
    for term in dnf:
        minterm = [0] * N
        for var in term:
            minterm[var] = 1
        minterms.append(minterm)
    return minterms

def compute_mean_width(minterms, N, num_samples=2000):
    total = 0.0
    for _ in range(num_samples):
        u = [random.gauss(0, 1) for _ in range(N)]
        norm = math.sqrt(sum(x*x for x in u))
        if norm == 0:
            continue
        u = [x/norm for x in u]
```

```
        max_dot = max(sum(u[i] * m[i] for i in range(N)) for m in minterms)
        total += max_dot
    return 2 * total / num_samples

def generate_random_dnf(s, N):
    dnf = []
    for _ in range(s):
        term_size = random.randint(1, N)
        term = random.sample(range(N), term_size)
        dnf.append(term)
    return dnf

def generate_clique_dnf(v, k):
    N = v * (v - 1) // 2
    dnf = []
    vertices = list(range(v))
    for subset in combinations(vertices, k):
        term = []
        for i, j in combinations(subset, 2):
            term.append(i * (v - 1) - i * (i - 1) // 2 + j - i - 1)
        dnf.append(term)
    return dnf

def run_trial(seed):
    random.seed(seed)
    metric_name = "mean_width_squared"
    metric_value = 0.0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    # Test random monotone DNFs
    for s in [5, 10, 20, 40]:
        for N in [12, 18, 24, 28]:
            dnf = generate_random_dnf(s, N)
            minterms = generate_minterms(dnf, N)
            mu = compute_mean_width(minterms, N)**2
            bound = 3 * math.log(s + 1) * (math.log(N) + 1)
            if mu > 2 * bound:
                conjecture_holds = False
                counterexample = f"Random DNF with s={s}, N={N} has mu={mu} > 2*bound={2*bound}"
                return {
                    "metric_name": metric_name,
                    "metric_value": mu,
                    "instances_tested": instances_tested + 1,
                    "conjecture_holds": conjecture_holds,
                    "counterexample": counterexample
                }
            metric_value += mu
            instances_tested += 1

    # Test k-CLIQUE indicators
    v_values = list(range(4, 10))
    mu_values = []
    for v in v_values:
        k = math.ceil(math.log2(v))
```

```
        dnf = generate_clique_dnf(v, k)
        N = v * (v - 1) // 2
        minterms = generate_minterms(dnf, N)
        mu = compute_mean_width(minterms, N)**2
        mu_values.append(mu)
        if mu < 0.5 * v:
            conjecture_holds = False
            counterexample = f"k-CLIQUE with v={v} has mu={mu} < 0.5*v={0.5*v}"
            return {
                "metric_name": metric_name,
                "metric_value": mu,
                "instances_tested": instances_tested + 1,
                "conjecture_holds": conjecture_holds,
                "counterexample": counterexample
            }
        instances_tested += 1

    # Check regression for k-CLIQUE
    if len(v_values) >= 2:
        n = len(v_values)
        sum_v = sum(v_values)
        sum_mu = sum(mu_values)
        sum_v_mu = sum(v * mu for v, mu in zip(v_values, mu_values))
        sum_v_sq = sum(v * v for v in v_values)

        slope = (n * sum_v_mu - sum_v * sum_mu) / (n * sum_v_sq - sum_v * sum_v)
        intercept = (sum_mu - slope * sum_v) / n

        r_squared = 1 - (sum((mu - (intercept + slope * v))**2 for v, mu in zip(v_values, mu_value
                         sum((mu - sum_mu / n)**2 for mu in mu_values))

        if slope < 0.5 or r_squared < 0.9:
            conjecture_holds = False
            counterexample = f"Regression for k-CLIQUE has slope={slope} < 0.5 or R²={r_squared} <

    return {
        "metric_name": metric_name,
        "metric_value": metric_value / instances_tested if instances_tested > 0 else 0.0,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["conjecture_holds"]]
    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_valid_trials")
        sys.exit(0)
```

```
mean = sum(metric_values) / len(metric_values)
std = math.sqrt(sum((x - mean)**2 for x in metric_values) / len(metric_values))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]),
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

Seed	Metric value	Holds?	Counterexample
11	1.1786757596570663	✗	k-CLIQUE with v=4 has mu=1.1786757596570663 < 0.5*v=2.0
23	1.174440820833285	✗	k-CLIQUE with v=4 has mu=1.174440820833285 < 0.5*v=2.0
37	1.1665762757636087	✗	k-CLIQUE with v=4 has mu=1.1665762757636087 < 0.5*v=2.0
53	1.1207983903354595	✗	k-CLIQUE with v=4 has mu=1.1207983903354595 < 0.5*v=2.0
71	1.144961123893079	✗	k-CLIQUE with v=4 has mu=1.144961123893079 < 0.5*v=2.0
89	1.1707937772957717	✗	k-CLIQUE with v=4 has mu=1.1707937772957717 < 0.5*v=2.0
103	1.1891971584829055	✗	k-CLIQUE with v=4 has mu=1.1891971584829055 < 0.5*v=2.0
127	1.1658549744593056	✗	k-CLIQUE with v=4 has mu=1.1658549744593056 < 0.5*v=2.0
149	1.1667559095219577	✗	k-CLIQUE with v=4 has mu=1.1667559095219577 < 0.5*v=2.0
167	1.1435334406905906	✗	k-CLIQUE with v=4 has mu=1.1435334406905906 < 0.5*v=2.0

Seed	Metric value	Holds?	Counterexample
191	1.1675978757962067	✗	k-CLIQUE with v=4 has mu=1.1675978757962067 < 0.5*v=2.0
211	1.1547431311751086	✗	k-CLIQUE with v=4 has mu=1.1547431311751086 < 0.5*v=2.0
233	1.182081204264698	✗	k-CLIQUE with v=4 has mu=1.182081204264698 < 0.5*v=2.0
257	1.1691016606560682	✗	k-CLIQUE with v=4 has mu=1.1691016606560682 < 0.5*v=2.0
277	1.1622877422239692	✗	k-CLIQUE with v=4 has mu=1.1622877422239692 < 0.5*v=2.0
311	1.1557196533341476	✗	k-CLIQUE with v=4 has mu=1.1557196533341476 < 0.5*v=2.0
347	1.1542473967654259	✗	k-CLIQUE with v=4 has mu=1.1542473967654259 < 0.5*v=2.0
389	1.146782435628744	✗	k-CLIQUE with v=4 has mu=1.146782435628744 < 0.5*v=2.0
421	1.136003662687513	✗	k-CLIQUE with v=4 has mu=1.136003662687513 < 0.5*v=2.0
463	1.1857866298955364	✗	k-CLIQUE with v=4 has mu=1.1857866298955364 < 0.5*v=2.0
503	1.1927318541604457	✗	k-CLIQUE with v=4 has mu=1.1927318541604457 < 0.5*v=2.0
547	1.1396899464107255	✗	k-CLIQUE with v=4 has mu=1.1396899464107255 < 0.5*v=2.0
593	1.1832069622303316	✗	k-CLIQUE with v=4 has mu=1.1832069622303316 < 0.5*v=2.0

Seed	Metric value	Holds?	Counterexample
631	1.1742179582193337	✗	k-CLIQUE with v=4 has mu=1.1742179582193337 < 0.5*v=2.0
677	1.1635078759135067	✗	k-CLIQUE with v=4 has mu=1.1635078759135067 < 0.5*v=2.0
727	1.1855075477194135	✗	k-CLIQUE with v=4 has mu=1.1855075477194135 < 0.5*v=2.0
773	1.172332125064841	✗	k-CLIQUE with v=4 has mu=1.172332125064841 < 0.5*v=2.0
821	1.1924667140957228	✗	k-CLIQUE with v=4 has mu=1.1924667140957228 < 0.5*v=2.0
877	1.1896142603341249	✗	k-CLIQUE with v=4 has mu=1.1896142603341249 < 0.5*v=2.0
929	1.1939804627587693	✗	k-CLIQUE with v=4 has mu=1.1939804627587693 < 0.5*v=2.0

Aggregate statistics:

Statistic	Value
n_seeds	30
metric_mean	1.1674398243422555
metric_std	0.01882432666165174
metric_ci95_half	0.006873672228288384
metric_min	1.1207983903354595
metric_max	1.1939804627587693
support_fraction	0.0

7. Test stdout (last 2KB)


```
me": "mean_width_squared", "metric_value": 1.136003662687513, "instances_tested": 17, "conjecture_ho
CLIQUE with v=4 has mu=1.136003662687513 < 0.5*v=2.0", "seed": 421}
TRIAL: {"metric_name": "mean_width_squared", "metric_value": 1.1857866298955364, "instances_tested"
CLIQUE with v=4 has mu=1.1857866298955364 < 0.5*v=2.0", "seed": 463}
TRIAL: {"metric_name": "mean_width_squared", "metric_value": 1.1927318541604457, "instances_tested"
CLIQUE with v=4 has mu=1.1927318541604457 < 0.5*v=2.0", "seed": 503}
TRIAL: {"metric_name": "mean_width_squared", "metric_value": 1.1396899464107255, "instances_tested"
CLIQUE with v=4 has mu=1.1396899464107255 < 0.5*v=2.0", "seed": 547}
TRIAL: {"metric_name": "mean_width_squared", "metric_value": 1.1832069622303316, "instances_tested"
CLIQUE with v=4 has mu=1.1832069622303316 < 0.5*v=2.0", "seed": 593}
```

```
TRIAL: {"metric_name": "mean_width_squared", "metric_value": 1.1742179582193337, "instances_tested"
CLIQUE with v=4 has mu=1.1742179582193337 < 0.5*v=2.0", "seed": 631}
TRIAL: {"metric_name": "mean_width_squared", "metric_value": 1.1635078759135067, "instances_tested"
CLIQUE with v=4 has mu=1.1635078759135067 < 0.5*v=2.0", "seed": 677}
TRIAL: {"metric_name": "mean_width_squared", "metric_value": 1.1855075477194135, "instances_tested"
CLIQUE with v=4 has mu=1.1855075477194135 < 0.5*v=2.0", "seed": 727}
TRIAL: {"metric_name": "mean_width_squared", "metric_value": 1.172332125064841, "instances_tested":
CLIQUE with v=4 has mu=1.172332125064841 < 0.5*v=2.0", "seed": 773}
TRIAL: {"metric_name": "
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The empirical data shows support_fraction = 0.0 — every single trial FALSIFIED the conjecture, yet it is being reviewed as if 'SUPPORTED.' Specifically, part (ii) fails: for v=4, μ ≈ 1.17, far below C_2·v with C_2=0.5 (i.e., 2.0). Even with C_2 much smaller, the metric μ ≈ MW^2 is bounded by the squared diameter of K(f) ⊆ [0,1]^N, and for small v the minterm hull is tiny — this is a metric-saturation / construction problem, not evidence. The lower bound μ ≥ C_2·v cannot hold for any fixed constant at

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: Every k-CLIQUE trial at v=4 yields μ ≈ 1.17, well below the falsification threshold 0.25·v = 1.0… actually 0.25·4=1.0 so this passes the 0.25·v floo | next: Rescale the lower bound to μ(f) ≥ C_2·v/log v or test the conjecture with normalized mean width MW(K(f))/√N to remove metric-saturation artifacts at small v.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	305414
2	propose	claude_max	opus	0	0	412510
3	preregistration	claude_max	opus	0	0	6852
4	novelty	claude_max	opus	0	0	3472
5	novelty	claude_max	opus	0	0	8433
6	test_gen	mistral	codestral-latest	0	0	311184
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	273170
8	test_gen	mistral	codestral-latest	0	0	309108
9	test_gen	mistral	codestral-latest	0	0	311180
10	critic	claude_max	opus	0	0	8649
11	judge	claude_max	opus	0	0	6355

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1956327 ms total latency. Provider mix: {'claude_max': 7, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in `research/audit/65d1ffdf3cc6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/65d1ffdf3cc6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/65d1ffdf3cc6.tar.gz` (if generated)